Ben Cuff

JSON Query Tool with AI Delegate and Review Process

repo diff transcript

For the project, I had AI spin up a basic JSON query CLI using Haskell. I chose Haskell because it's not a super common language and AI tends to struggle with its types and tends to be a bit hacky in its implementation, rather than following idiomatic language conventions. It tends to be a white paper language, so there is significantly less training data than for js or python for example. Plus I find the language interesting. To begin with, the JSON parser is only capable of keying into 1 value, either based on the record name or index (using dot syntax or array index). The harness I used is opencode and the AI model I used is *DeepSeek V4 Flash Free*. The new feature I chose to add is filtering, counting, mapping, and sorting. I had the AI organize its progress into individual diffs for each new feature. All development occurred on a feature branch (dev). After the review comments were addressed and acceptance tests passed, I merged the branch into main. The first checkpoint I decided on was a planning period where I read the AI's plan for implementing the new features and give feedback and iterate before finally signing off on a proposed implementation. The second checkpoint is where I read and review the code myself. I did this in the form of github comments on the actual diff. The AI was instructed to use the github CLI to read the comments as well as for submitting their own commits/comments (with a trailer attributing it to AI). After the second checkpoint, the AI will fix the review comments and make a final commit. Then, I can do a quick look and merge the diff. These checkpoints allowed me to first define the basic business logic of the new features, and then improve the code quality and hygiene of the code.

Acceptance Tests

	input	output
Count empty array	[] count	0
Count elements	[{ "id": 1 }, { "id": 2 }, { "id": 3 }, { "id": 4 }] count	4
Filter based on number	[{ "name": "Alice", "age": 22 }, { "name": "Bob", "age": 19 }, { "name": "Charlie", "age": 25 }] Filter .age > 20	[{ "name": "Alice", "age": 22 }, { "name": "Charlie", "age": 25 }]
Filter based on string	[{ "name": "Alice", "major": "CS" }, { "name": "Bob", "major": "Math" }, { "name": "Charlie", "major": "CS" }] Filter .major == “CS”	[{ "name": "Alice", "major": "CS" }, { "name": "Charlie", "major": "CS" }]

Initial Prompt

“Your job is to implement the following JSON query capabilities for the current project.

Filtering, mapping, counting, and sorting. Start by planning first and then waiting for my signoff

before beginning your implementation. Be sure to look at the current state of the project first before implementing. Then when you are ready to start writing code, once you finish, open a diff via gh (the github CLI) . After this I will review the code in github and you will look at those comments and fix them as I tell you to. Please also organize into multiple commits, 1 for each feature. When you commit, also be sure to append "Assisted by AI" or whatever the standard is for github commits, there should be some nice way to do this. Make no mistakes."

Reflection

My first checkpoint was a planning period where I gave the AI an initial prompt and had it ask me questions about certain business logic and implementation details. These questions that the AI asked helped iron out some of the complexity without giving the agent too much agency, no pun intended. The planning caught 3 small things, nothing really super glaring came up though. The first was projects, in which I defined the syntax that I wanted the AI to use for its JSON projections. I also told the AI how to handle sorting with incompatible and ambiguous types. The last thing that I caught was the syntax for chaining operations. I decided that you should be able to do spaces or pipes (|) to chain operations. The second checkpoint was the code review checkpoint. I ran all four acceptance tests after the implementation and they passed. This allowed me to correct some code quality issues and make high level code organization decisions. First, I had the AI add some edge case test coverage and separated the tests into multiple files (which is the standard for Haskell tests). Then, I told the AI to treat different types as not equal instead of equal when comparing. Filtering and sorting also did not allow for additional whitespace support. Finally, I caught an opportunity to dedupe some code when parsing, making the code more DRY and easier to extend in the future. I didn't see anything crazy from the AI in terms of scope creep. I designed the initial prompt well and the planning period helped make

sure this was well defined. Also, there are a ton of training resources online that the AI is likely trained on, so it is unlikely to make any weird scope creep decisions. I also didn't see any test gaming, as Haskell in general is pretty good at preventing type coercion and makes it hard to game tests by having very strong types. It did, however, often test the "happy" path and miss some of the edge cases in its test, but I had it add more test coverage during the review stage. The AI did not make any confident wrong turns that I noticed. Reviewing the AI's code feels similar to reviewing teammates, but I need to have a closer eye as I am less likely to blindly trust that the AI has reviewed and tested the code. I need to understand what every part of the code does before signing off on it. AI may accelerate the actual writing of code, but it has not completely automated auditing and ownership over the code. Ultimately, if the code I ship causes a sev, then it's my fault and not the AI, as the AI has no ownership over the code it generates. I think this workflow in general works pretty well, but I do think there needs to be another person to sign off on the code. Also, it could help to have a code review agent that reviews the code before I do, to catch the low hanging fruit. This follows the general workflow I use at my internship. First, I have the AI write all of the code. Then, I have the internal code reviewer take a look and have the AI fix the feedback. Then, I read the code myself and manually verify that it works as expected. Finally, I publish the diff for my teammates to take a look at. Someone else should not be the first one to read the code that you generate via AI. This overloads the verification process onto someone else, wasting their time, and making them less likely to review your code in the future. Also, I think that the diff description should be written manually, as the AI often writes far too much and too much detail without including the core goal behind the diff.